Programlama Dilleri

Dr. Öğr. Üyesi Hayri Volkan Agun

Bilgisayar Mühendisliği Bölümü

Bursa Teknik Üniversitesi

İfadeler ve Atama

- İfadeler (expressions), programlamada hesaplamaları tanımlamanın temel yoludur.
  - İki önemli boyut:
    - Syntax (sözdizimi)
    - Semantics (anlambilim)
- Önemli kavramlar:
  - Operatör önceliği (precedence)
  - İlişkisellik (associativity)
  - Operand değerlendirme sırası
    - Emperatif dillerde atama (assignment) program durumunu değiştirir.
    - Fonksiyonel diller: Yan etki yoktur. Daha öngörülebilir bir sonuç beklenir.

- **Aritmetik İfadelerin Yapısı**
- Bileşenler:
  - Operatörler
  - Operandlar
  - Parantezler
  - Fonksiyon çağrıları
- Operatör türleri:
  - Unary → -a
  - Binary → a + b
  - Ternary → a ? b : c

```
x = a + b * c
```

  - LISP

```
(+ a (* b c))
```

İfadeler ve Atama

- Genel öncelik:
  - Üs alma (**)
  - Çarpma / Bölme
  - Toplama / Çıkarma

```
3 + 4 * 5 = 23
(3 + 4) * 5 = 35
```

- İlişkisellik
  - Aynı öncelikteki operatörlerin sırası
  - 📌 **Sol ilişkili:**
    - a - b + c → (a - b) + c
  - 📌 **Sağ ilişkili (üs alma):**
    - 2 ** 3 ** 2 → 2 ** (3 ** 2) = 512
  - 📌 **Ada dili örneği:**
    - A ** B ** C → HATA
      (A ** B) ** C veya A ** (B ** C)

- **Parantez Kullanımı**
  - Önceliği **manuel olarak belirler**
  - 📌 **Örnek:**
    - A + B * C
      (A + B) * C
  - Parantez:
    - Okunabilirliği artırır
    - Hataları azaltır
  - 📌 **Örnek (overflow önleme):**
    - (A + B) + (C + D)
    - A / B * B –» A * B / B

İfadeler ve Atama

- Operand sırası çoğu dilde belirsizdir.
- 📌 Örnek:
  - a = 10
  - b = a + fun(a)
  - Eğer fun(a) → a'yı değiştirirse:
    - önce a b=20
    - önce fun(a) b=30

```c
int a = 5;
int fun1() {
  a = 17;
  return 3;
}
a = a + fun1();
```

Sonuç:8 veya 20 olabilir ❗

Referential Transparency ve Atama

- Aynı değere sahip ifadeler birbirinin yerine geçebiliyorsa
- 📌 **Örnek:**
  - result1 = (fun(a) + b) / (fun(a) - c);
    temp = fun(a);
    result2 = (temp + b) / (temp - c);
- Eğer fun yan etki içerirse (b veya c değerine 1 eklemiş olsun.):
  - result1 ≠ result2
- ✓ Fonksiyonel diller:
  - Yan etki yok
  - Daha kolay analiz edilir

Operatör Aşırı Yükleme

- Aritmetik operatörler genellikle birden fazla amaç için kullanılır.
- Örnek: + operatörü
  - Tamsayı toplama
  - Ondalıklı sayı toplama
  - String birleştirme (örneğin Java)
- Bu duruma operatör aşırı yükleme (operator overloading) denir.
  - ✓Avantaj: Doğru kullanıldığında okunabilirliği artırır
  - Dezavantaj:Yanlış veya aşırı kullanım anlaşılabilirliği ve güvenilirliği azaltır.

- C++ dilinde & operatörü iki farklı anlamda kullanılır:
  - Binary (iki operandlı):
    - Bit düzeyinde AND işlemi
    - Örnek: a & b
  - Unary (tek operandlı):
    - Değişkenin adresini verir (address-of operator)
    - Örnek:
    - x = &y;
- ❗ Problemler:
  - Aynı sembolün farklı anlamlara gelmesi → okunabilirliği düşürür.
  - Yazım hatası (operand eksikliği) → derleyici tarafından fark edilmeyebilir .
  - Hatalar tespiti zor hale gelir.

Operatör Aşırı Yükleme

- Bazı diller: C++ C# F#
  - Kullanıcıların operatörleri yeniden tanımlamasına izin verir.
  - ✓Örnek:* operatörü: Bir skaler ile dizi çarpımı olarak tanımlanabilir
  - ✓Avantaj:Kod daha okunabilir hale gelir.
  - ✓ Örnek karşılaştırma:
    - A * B + C * D
    - Add(Mult(A, B), Mult(C, D))
- Kullanıcı + operatörünü çarpma olarak tanımlayabilir.
- Operatörün anlamı:Operand türüne bağlıdır.
- Başka dosyalarda tanımlı olabilir.
  - Modüler sistemlerde problem: Farklı ekipler aynı operatörü farklı anlamda kullanabilir.
  - Kısıtlamalar:C++ bazı operatörleri overload etmeye izin vermez:. (member access):: (scope resolution)
  - 📌 Not:Operatör aşırı yükleme:Java’da yoktur. Ama C#’ta tekrar eklenmiştir

Tür Dönüşümleri - Widening ve Narrowing

- **Widening (Genişletme)**
  - Değerin daha büyük kapsama sahip bir türe dönüştürülmesi
  - ✓ Genellikle güvenlidir (değer kaybolmaz)
  - Örnek: int → float (Java)
- **Narrowing (Daraltma)**
  - Daha küçük kapsama sahip türe dönüşüm
  - ❗ Veri kaybı olabilir
  - Örnek: double → float
  - Örnek hata: 1.3E25 → int → tamamen farklı sonuç.
  - Hassasiyet kaybı (precision loss)

- **Coercion ve Mixed-Mode İfadeler**
- **Coercion**
  - Derleyici tarafından otomatik yapılan örtük tür dönüşümü
- **Mixed-mode expression**
  - Farklı türler bir arada kullanılabilir
  - Örnek Java:
  - int a; float b, c, d;
    d = b * a; // a → float'a çevrilir
  - Yanlış operand kullanımı derleyici hatası vermez.
- Dil tasarım farklılıkları:
  - Ada → çok katı, neredeyse mixed-type yok.
  - F#, ML → explicit dönüşüm gerekli.
  - Java → esnek ama hata riski var.

Tür Dönüşümleri

- Explicit Conversion ve Hatalar
- Programcı tarafından açık şekilde yapılır.
  - C tabanlı diller:(int) angle
  - F# / ML: Fonksiyon şeklinde
    - float(sum)
- İfade hataları:
  - Overflow / Underflow → değer çok büyük/küçük.
  - Division by zero → sıfıra bölme.
  - Bunlar run-time errors / exceptions.

Boolean İfadeler

- Relational expressions
  - Sonuç: Boolean (true/false)
  - Operatörler: ==, !=, <, >, <=, >=
  - Tür farklılıklarına göre operatör overload edilebilir
  - JavaScript özel durumu:
    - "7" == 7 // true (coercion var)
    - "7" === 7 // false (coercion yok)
    - Boolean expressions
      - AND, OR, NOT
      - Öncelik: C tabanlı diller → AND > OR
      - Ada → AND = OR önceliği
      - C'nin ilginç özelliği:a > b > c
        - Soldan sağa değerlendirme → beklenmeyen sonuç
    - ✓ Boolean türü olan diller → daha iyi hata tespiti

Kısa devre değerlendirme

- Tüm operandları/işleçleri değerlendirmeden sonucu belirleme.
- Örnek (aritmetik değil):
  - (a >= 0) && (b < 10)
  - a < 0 → ikinci ifade değerlendirilmez.
- Gereksiz hesaplamalardan kaçınır.
- Risk: Eğer kısa devre kullanılan ifade yan etkiler içeriyorsa, beklenen değişiklik gerçekleşmeyebilir.
  - Örnek: a > b || (b++) / 3

- Ada → and then, or else (short-circuit), and, or (non-short-circuit)
- C tabanlı diller → &&, || short-circuit, &, | non-short-circuit
- Ruby, Perl, ML, F#, Python → tüm mantıksal operatörler short-circuit

Atamalar

- Basit Atama (Simple Assignment)
- Atama operatörü: Genellikle = kullanılır
  - ALGOL 60 → := kullanmış, Ada da benzer
- Amaç: Değeri değişkenle ilişkilendirmek
- Dil tasarım farklılıkları:
  - Fortran, Ada → atama tek başına, tek değişken
  - Perl → koşullu hedefler mümkün:
    - ($flag ? $count1 : $count2) = 0;

- Bileşik ve Unary Atama Operatörleri
- Bileşik atama (Compound assignment):
  - a = a + b → a += b
  - Kapsamlı diller: C, JavaScript, Python, Perl, Ruby
- Unary atama (Artırma / Azaltma):
  - ++ → artırma, -- → azaltma
  - Prefix: önce artırılır, sonra değer kullanılır
    - sum = ++count;
  - Postfix: önce değer kullanılır, sonra artırılır
    - sum = count++;

Atamalar

- Atama İfadesi ve Çoklu Hedef Atama
- Atama bir ifade olarak kullanılabilir:
  - while ((ch = getchar()) != EOF) { ... }
  - Değer ataması sonucu, diğer işleçlerle karşılaştırılabilir.
  - Yan etkiler:Okunması ve anlaşılması zor ifadeler oluşabilir.
    - a = b + (c = d / b) - 1;
    - Çoklu hedef atama:Perl / Ruby / Lua:
      - ($first, $second, $third) = (20, 40, 60);
      - ($first, $second) = ($second, $first);
        - // değer takası.
- Fonksiyonel Dillerde Atama
  - Değişkenler sabit değer isimleri (immutable)
    - ML → val cost = quantity * price;
    - Yeni val → eskiyi gizler, yeni sürüm oluşturur
    - F# → let kullanır.
      - let yeni scope oluşturur.
      - val → scope değiştirmez.
- Mixed-Mode Atama
- Sorun: Atanan değişken türü ile ifade türü farklıysa ne yapılır?
  - C, C++, Perl: Coercion ile izin verilir (type cast)
  - Ada: Mixed-mode atama yasak
  - Java, C#: Sadece widening (genişletme) izinli
  - int → float ✅ float → int ❌
  - Fonksiyonel diller: Atama yok, sadece değer isimlendirme.

Sorular

- Coercion (Zorunlu Tür Dönüşümü) nedir?
- Koşullu ifade (conditional expression) nedir?
- Aşırı yüklenmiş operatör (overloaded operator) nedir?
- Daraltan (narrowing) ve genişleten (widening) dönüşümleri tanımlayınız.
- Referans şeffaflığı (referential transparency) nedir?
- Referans şeffaflığının avantajları nelerdir?
- Operand değerlendirme sırası (operand evaluation order) fonksiyonel yan etkilerle nasıl etkileşir?
- Short-circuit değerlendirme (short-circuit evaluation) nedir?